

(An Autonomous Institution, Affiliated to Anna University, Chennai)

CROPTAG – GRAIN, MILLET AND PULSE IDENTIFICATION USING CNN

Submitted by

ARUNTHIRAVIAM M

2116241501023

ATCHAYA R

2116241501026

AI23531 DEEP LEARNING

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam

Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

NAME	ARUNTHIRAVIAM M ATCHAYA R		
ACADEMIC YEAR	2026 – 2027	**SEMESTER**	V
BRANCH	Artificial Intelligence and Machine Learning		

UNIVERSITY REGISTER NO.	2116241501023 2116241501026

Certified that this is the bonafide record of work done by the above student in the Mini Project titled **CropTag – Grain, Millet and Pulse Identification using CNN** in the subject **AI23531 Deep Learning** during the year **2026 - 2027**.

Signature of Faculty – in – Charge

Submitted for the Practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Grains, millets and pulses are staple foods, yet many varieties look very similar and information about their nutritional value is rarely available at the point of use. CropTag is a deep learning-based web application that identifies 38 varieties of grains, millets and pulses from a single photograph using a custom Convolutional Neural Network (CNN) trained from scratch on a labeled image dataset. The trained V2 model is served through a local Flask inference server, so images are processed on the user's own machine.

The web interface allows the user to upload or drag and drop a JPG or PNG image, preview it and press Analyze. The image is preprocessed and passed to the CNN, which produces a score for each of the 38 classes. The class with the highest score is displayed as the predicted crop together with its category, a confidence percentage and its nutrition profile (energy, protein, carbohydrates, fiber and fat per 100 g). The result page also presents health benefits, common culinary uses, a short fact and related varieties from the same category.

CropTag demonstrates the practical use of deep learning and computer vision in food and agriculture by providing an accessible, locally run platform for crop identification and nutrition awareness.

Keywords:

Deep Learning, Convolutional Neural Network, Crop Identification, Grains, Millets, Pulses, Image Classification, Flask, Nutrition Information, Computer Vision.

TABLE OF CONTENTS

CH. NO.	TITLE	PAGE NO.
ABSTRACT		III
1	INTRODUCTION	1
	1.1 INTRODUCTION	1
	1.2 OBJECTIVES	1
	1.3 SCOPE OF THE PROJECT	2
2	LITERATURE REVIEW	3
	2.1 DEEP LEARNING FOR IMAGE CLASSIFICATION	3
	2.2 CNNs FOR CROP AND FOOD RECOGNITION	3
	2.3 IMAGE DATASET	3
	2.4 RESEARCH GAP	4
3	SYSTEM REQUIREMENTS	5
	3.1 HARDWARE REQUIREMENTS	5
	3.2 SOFTWARE REQUIREMENTS	5
	3.3 LIBRARIES USED	5
4	SYSTEM OVERVIEW	6
	4.1 EXISTING SYSTEM	6
	4.1.1 DRAWBACKS OF EXISTING SYSTEM	6
	4.2 PROPOSED SYSTEM	6
	4.2.1 ADVANTAGES OF PROPOSED SYSTEM	6
5	SYSTEM IMPLEMENTATION	8
	5.1 SYSTEM ARCHITECTURE DIAGRAM	8
	5.2 SYSTEM FLOW	9
	5.3 LIST OF MODULES	10
	5.4 MODULE DESCRIPTION	10
6	RESULTS AND DISCUSSION	13
	6.1 IMPLEMENTATION RESULTS	13
	6.2 RESULTS DISCUSSION	14
	6.3 PERFORMANCE EVALUATION	14
	6.4 LIMITATIONS	14
	6.5 CONCLUSION	15
	6.6 FUTURE ENHANCEMENTS	15
7	APPENDIX	16
	7.1 SAMPLE CODE	16
	7.2 OUTPUT SCREENSHOTS	17
	REFERENCES	21

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Grains, millets and pulses form the backbone of the daily diet in India and many other parts of the world. They supply energy, plant protein, dietary fiber and essential minerals, and they are used in an enormous variety of dishes. Millets in particular have received renewed attention in recent years as nutritious and climate-resilient crops.

Despite their importance, correctly recognizing a particular variety is not always easy. Many pulses and dals have a similar size, shape and color once they are hulled or split, and several millets look alike to anyone without hands-on experience. Consumers, small retailers and students therefore often struggle to name what they are looking at, and the nutritional value of each variety is rarely available at the moment it is needed.

Traditionally, identification depends on the experience of farmers, traders and cooks, who judge a sample by its color, size, shape and surface texture. This knowledge is valuable, but it is personal, difficult to transfer and not always consistent from one observer to another.

Deep learning and computer vision offer an alternative. Convolutional Neural Networks (CNNs) are widely used for image classification because they learn visual features such as edges, textures, shapes and color patterns directly from labeled images, without requiring hand-crafted features.

This project, CropTag, applies a custom-built CNN, trained from scratch on a labeled image dataset, to identify 38 varieties of grains, millets and pulses from a single photograph. The trained V2 model is served through a local Flask inference server. A web interface lets the user upload a photo, press Analyze and immediately see what the crop is, how confident the model is, and its nutrition profile.

The project demonstrates how a deep learning model can be combined with a simple web application to turn raw image classification into a practical, informative tool for everyday kitchens.

1.2 OBJECTIVES

The main objectives of the project are:

- To develop an image-based crop identification application using deep learning.

- To design and train a custom CNN from scratch to classify 38 varieties of grains, millets and pulses.
- To preprocess uploaded images into the standard format required by the model.
- To serve the trained model through a local Flask inference API.
- To display the predicted crop, its category and a confidence score.
- To present nutrition information (energy, protein, carbohydrates, fiber and fat per 100 g), health benefits, common uses and related varieties for each predicted crop.
- To build a clean, user-friendly web interface with image upload, drag-and-drop and image preview.
- To demonstrate the application of computer vision in food and agriculture.

1.3 SCOPE OF THE PROJECT

The project focuses on image-based identification of 38 varieties of grains, millets and pulses using a trained CNN, and on presenting the related nutrition information through a web interface.

The application accepts JPG and PNG images and works best with a photograph of a single item or a single type of grain. It runs locally: the uploaded image is processed by the project's own Flask inference server on the user's computer.

The current implementation is an academic prototype. It does not assess quality, grade, freshness or adulteration, and it cannot identify varieties outside the 38 classes on which it was trained. The nutrition values shown are indicative reference values per 100 g and may differ from those of a specific sample. Predictions depend on the trained model and on the quality of the input image.

CHAPTER 2
LITERATURE REVIEW

2.1 DEEP LEARNING FOR IMAGE CLASSIFICATION

Deep learning has become the dominant approach to image classification because it learns useful features directly from data. Earlier approaches relied on manually designed features such as color histograms, shape descriptors and texture measures, which were then passed to a conventional classifier. Deep networks instead learn a hierarchy of representations during training, from simple edges in the early layers to more complex patterns in deeper layers.

CNNs are especially suitable for images because convolution filters capture local patterns and share weights across the whole image. This keeps the number of parameters manageable and makes the model tolerant to small shifts in the position of objects.

2.2 CNNs FOR CROP AND FOOD RECOGNITION

Crop, seed and food recognition share a common challenge: classes are often separated by subtle visual cues. Grains, millets and pulses differ mainly in color, size, shape, surface texture and whether they are whole or split, and several classes are visually close to one another.

A typical CNN consists of convolutional layers, activation functions, pooling layers and fully connected classification layers. During training the network adjusts its weights to associate visual patterns with class labels. The final layer produces one score per class, and a softmax activation converts these scores into probabilities that sum to one.

CropTag uses a custom CNN trained from scratch. Training from scratch means that all weights are learned from the project's own dataset rather than being initialized from a model pre-trained on a large generic dataset. This gives complete control over the architecture and keeps the model compact enough to run on an ordinary computer, but it relies on a sufficiently varied dataset and on careful training to avoid overfitting.

2.3 IMAGE DATASET

The model is trained on a labeled image dataset in which every image belongs to one of 38 grain, millet or pulse classes. Kaggle URL: https://www.kaggle.com/datasets/arunthiraviam/indian-grains-pulses-and-millets-dataset

Dataset quality strongly influences the final model. Images that vary in lighting, background, camera, viewing distance and quantity of grains help the model generalize, whereas images

captured under uniform conditions can lead to a model that performs well on similar images but poorly on everyday photographs.

The application itself reflects this limitation: the upload screen advises that a photo of a single item works best.

2.4 RESEARCH GAP

Many image classification projects stop at displaying a class label. CropTag extends the classification result into practical information: after identifying the crop, the application presents its category, nutrition profile, health benefits, common culinary uses and related varieties on one screen. The project focuses on building an accessible, locally run application around a custom CNN rather than on proposing a new network architecture.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

- Processor: Modern Intel or AMD processor.
- RAM: 8 GB minimum; additional memory is beneficial, especially for training.
- Storage: Sufficient free space for the project files, Python environment, dataset and trained model.
- Camera: Optional; a phone or webcam can be used to take photos that are then uploaded.
- GPU: Helpful for training the CNN; not required for inference with the trained model.
- Display: Standard monitor or laptop display.

3.2 SOFTWARE REQUIREMENTS

- Operating System: Windows 10 or later (Linux and macOS are also suitable).
- Programming Language: Python.
- Development Environment: Visual Studio Code or any Python IDE.
- Deep Learning Framework: TensorFlow
- Backend Framework: Flask (local inference API).
- Frontend: HTML, CSS and JavaScript (dark-themed web interface).
- Web Browser: Any modern browser such as Chrome, Edge or Firefox.
- Dataset: Labeled image dataset of 38 grain, millet and pulse varieties.
- Trained Model: CropTag V2 model.

3.3 LIBRARIES USED

Library / Technology	Purpose
TensorFlow	Building, training, saving and loading the CNN model
Flask	Serving the trained model through a local inference API and hosting the web pages
Numpy	Numerical operations and array preparation
Pillow	Opening, converting and resizing uploaded images
HTML / CSS / JavaScript	Building the web interface, image preview, drag-and-drop upload and result display

Table 3.1: Libraries Used

CHAPTER 4
SYSTEM OVERVIEW

4.1 EXISTING SYSTEM

Traditional identification of grains, millets and pulses relies on manual observation. Farmers, traders and cooks examine the color, size, shape and texture of a sample and compare it with what they have seen before. Reference charts, books and general internet searches may be used to look up a variety and its nutritional value separately.

Manual identification can be reliable in experienced hands, but it depends on personal knowledge and does not scale to students, new consumers or anyone unfamiliar with regional varieties. Nutrition information must be searched for in a separate step.

4.1.1 DRAWBACKS OF EXISTING SYSTEM

- Identification depends on personal experience and is not easily shared.
- Visually similar varieties, especially split pulses, are easy to confuse.
- Results can vary between observers.
- Nutrition information is not available at the point of identification.
- Searching books or websites for every unknown sample is slow.

4.2 PROPOSED SYSTEM

CropTag is a deep learning-based web application that identifies a grain, millet or pulse from a photograph. It uses a custom CNN, trained from scratch, to classify an image into one of 38 varieties, and a local Flask server to run the trained V2 model.

The user opens the web interface, uploads or drags and drops a JPG or PNG image, checks the preview and clicks Analyze. The server converts the image to RGB, resizes and normalizes it, and passes it to the CNN. The model produces a score for each of the 38 classes; the class with the highest score is selected as the prediction.

The application then displays the predicted crop name, its category (for example Pulse), a confidence bar with the confidence percentage, and the nutrition profile per 100 g. It also shows a short description, three health benefits, common culinary uses, a "Did you know?" fact and the related varieties of the same category, with the predicted variety highlighted.

The system runs locally on the user's computer and does not require the model to be retrained for each prediction.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Provides automatic identification of 38 varieties of grains, millets and pulses from a single photo.
- Combines identification with nutrition information in one step.
- Shows a confidence score with every prediction.
- Offers a simple upload, preview and Analyze workflow with drag-and-drop support.
- Runs locally through a Flask inference server; the model is loaded once and reused.
- Presents benefits, uses and related varieties to support informed food choices.
- Demonstrates a from-scratch computer vision project end to end, from model to web interface.

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE DIAGRAM

The CropTag system consists of a web interface, an image upload stage, a Flask inference server that contains the preprocessing stage and the trained CNN, a nutrition information lookup, and a results display.

The process begins when the user uploads an image through the web interface and presses Analyze. The image is sent to the Flask server, converted to RGB format, resized to the input size of the model and normalized. The trained CNN generates scores for 38 classes, and the class with the highest score is selected. The server attaches the nutrition information of the predicted class, and the browser displays the result.

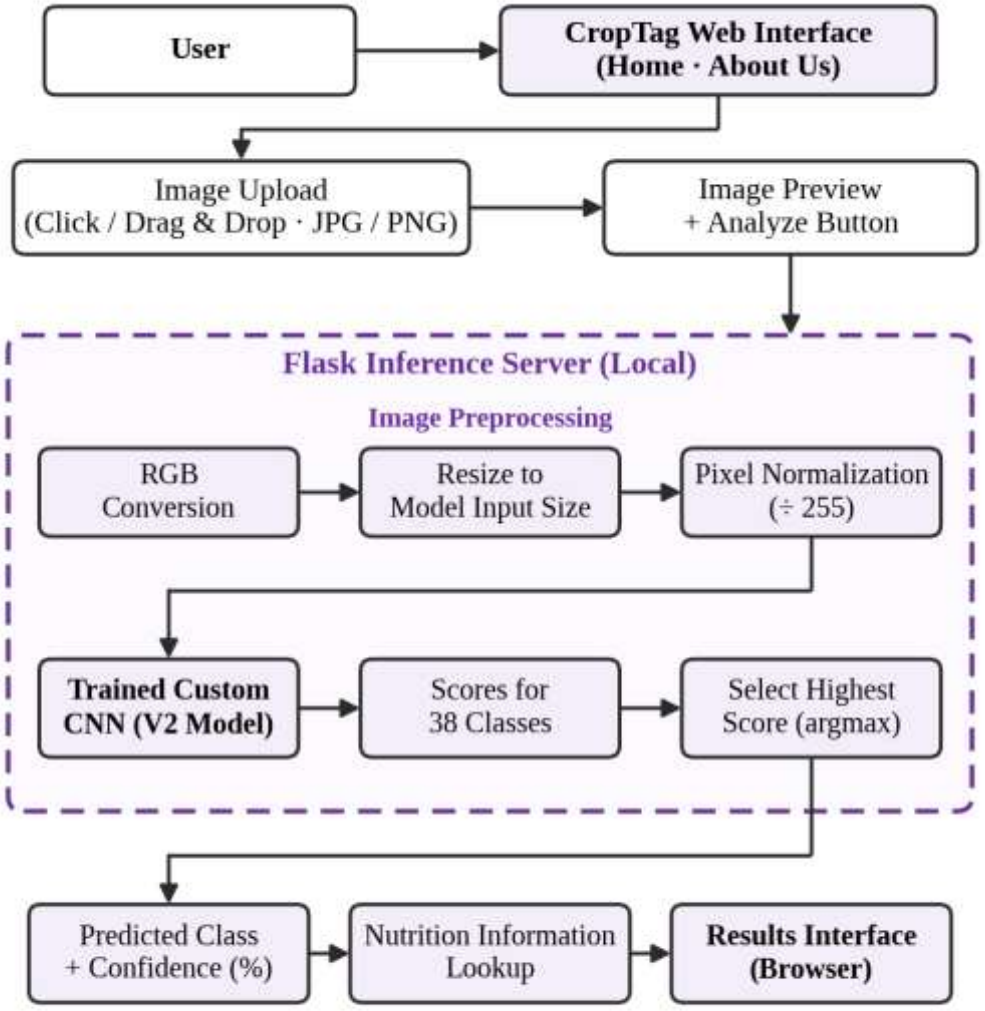


Fig. 5.1: System Architecture

5.2 SYSTEM FLOW

The system follows a sequence of operations to identify the crop in the supplied image.

1. The user opens the CropTag web application.
2. The user uploads, or drags and drops, a JPG or PNG image.
3. The image preview is displayed and the user clicks Analyze.
4. The image is sent to the local Flask inference server.
5. The image is converted to RGB format and resized.
6. Pixel values are normalized to the range 0–1.
7. The trained CNN (V2 model) processes the image.
8. The model generates scores for the 38 classes.

9. The class with the highest score is selected, along with its confidence.
10. The nutrition information is retrieved and the result is displayed.

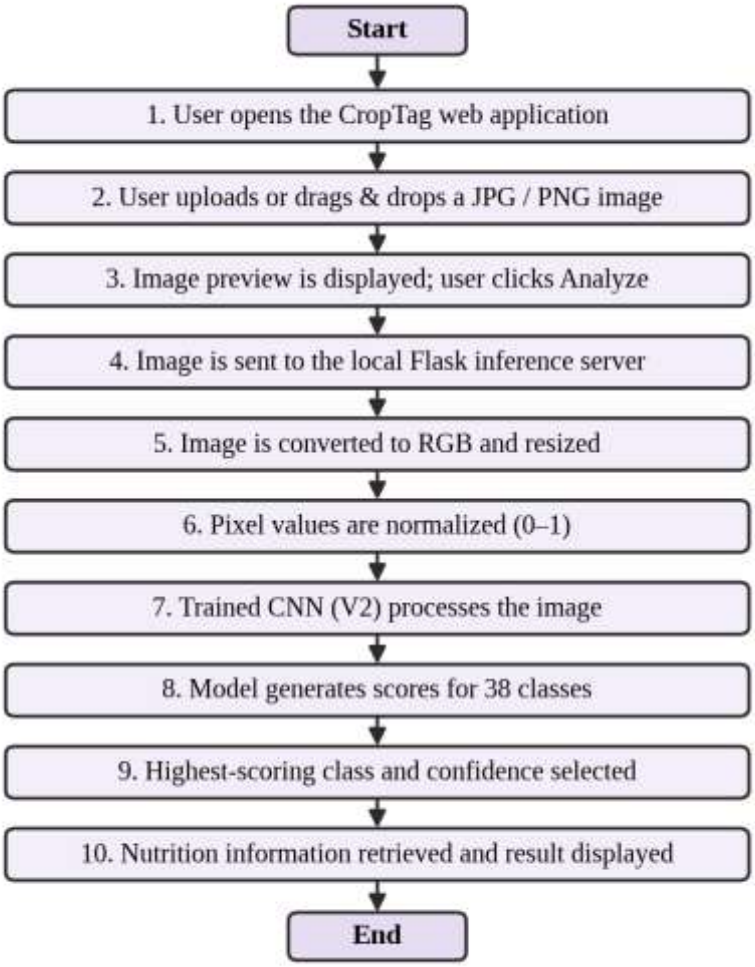


Fig. 5.2: System Flow Diagram

5.3 LIST OF MODULES

The application consists of the following modules:

- Web Interface Module
- Image Input and Preview Module
- Flask Inference API Module
- Model Loading Module
- Image Preprocessing Module
- Crop Classification Module
- Prediction and Confidence Module
- Nutrition Information Module

5.4 MODULE DESCRIPTION

5.4.1 Web Interface Module

The Web Interface Module provides the pages that the user sees. It has a navigation bar with the CropTag logo and the Home and About Us links, in a dark green theme. The Home page carries the heading "Identify Grains, Millets & Pulses Instantly" and an upload area. The About Us page explains the project through three cards: What is this?, How it works and Why it matters.

5.4.2 Image Input and Preview Module

This module lets the user supply an image by clicking the upload area or by dragging and dropping a file onto it. JPG and PNG images are supported, and the interface advises that a photo of a single item works best. After selection, the image is previewed with two buttons: Analyze, which starts the prediction, and Change Image, which allows another image to be chosen.

5.4.3 Flask Inference API Module

The Flask server receives the image from the browser, passes it through preprocessing and the CNN, and returns the prediction, confidence and nutrition information to the interface. Because the server runs locally, the image is processed on the user's own computer.

5.4.4 Model Loading Module

This module loads the trained V2 model into memory when the server starts. The model is loaded once and reused for every request, so it does not have to be retrained or reloaded for each prediction.

5.4.5 Image Preprocessing Module

The Image Preprocessing Module prepares the uploaded image in the format expected by the CNN. The image is converted to RGB format and resized to the fixed input size of the model. The pixel array is normalized by dividing the values by 255, and a batch dimension is added before the array is passed to the model.

5.4.6 Crop Classification Module

The Crop Classification Module uses the custom CNN, trained from scratch, to classify the preprocessed image. The network follows the general structure described in Chapter 2: convolutional layers with activation functions to extract visual features, pooling layers to reduce spatial size, and fully connected layers that end in an output layer with 38 units, one for each supported crop class.

Model Architecture

CropTag V2 uses a custom Convolutional Neural Network (CNN) with residual connections for recognizing 38 classes of Indian grains, millets, and pulses.

Layer Details

- **Input shape:** 160 × 160 × 3 (RGB image)
- **Initial convolution:** 32 filters
- **Residual blocks:** 7 blocks with filter sizes of 32, 64, 64, 128, 128, 192, and 192
- **Final convolution:** 256 filters
- **Normalization:** Batch Normalization
- **Activation function:** Swish
- **Pooling:** Global Average Pooling (GAP)
- **Dense layer:** 256 units with Swish activation
- **Dropout rate:** 0.35
- **Output heads:**
    - Class output: 38 classes
    - Category output: 3 categories — Grain, Millet, and Pulse

Training Configuration

- **Batch size:** 32

- **Optimizer:** AdamW
- **Initial learning rate:** 0.0003
- **Weight decay:** 0.0001
- **Data augmentation:** Dynamic on-the-fly augmentation
- **Learning-rate scheduler:** ReduceLROnPlateau
- **Early stopping:** Monitors validation class accuracy

Model Summary

The architecture uses residual connections to improve feature learning and Global Average Pooling to reduce the number of parameters. The two output heads support fine-grained classification and broader category prediction.

To obtain the exact total number of layers and trainable parameters, run model.summary() on the saved V3 model. The exact output should be added here rather than estimated.

5.4.7 Prediction and Confidence Module

This module interprets the output of the CNN. The index of the highest output score is found using the argmax operation and is mapped to the corresponding class name in the predefined label list. The highest score is displayed as a percentage on a confidence bar. This value is the model's output confidence and should not be read as a guarantee that the prediction is correct.

5.4.8 Nutrition Information Module

After the crop is identified, this module supplies the information associated with the predicted class so that the result page can show it. Table 5.1 lists the items displayed.

Item	Description	Example (from the application)
Crop name	Predicted variety	Whole White Gram
Category tag	Grain, millet or pulse group of the predicted crop	Pulse
Confidence	Highest model score shown as a bar and percentage	98%
Nutrition tiles	Energy, protein, carbohydrates, fiber and fat per 100 g	341 kcal, 25 g, 58 g, 18 g, 1.4 g

Item	Description	Example (from the application)
About	Short description of the crop	Nutrient-dense pulse valued in traditional Indian cuisine
Benefits	Three benefit cards	Protein Power, Digestive Health, Heart Health
Uses	Common culinary uses	Dal curries & sambar, Sprouts & salads, Soups & stews, Vadas, chaats & snacks
Did you know?	Short highlighted fact	High protein and fiber, good for gut health
Related varieties	Other varieties in the same category, with the predicted one highlighted	Black Gram, Green Gram, Rajma, Whole White Gram, etc.

Table 5.1: Information Displayed on the Result Page

CHAPTER 6
RESULTS AND DISCUSSION

6.1 IMPLEMENTATION RESULTS

The CropTag application was executed successfully in a local environment. The Flask inference server loaded the trained V2 model, and the web interface was displayed in a browser with the Home and About Us pages. Table 6.1 summarizes the behavior observed during testing.

Feature Tested	Observed Result
Application launch (Flask server and web interface)	Successful
Home page with upload area (click or drag & drop, JPG / PNG)	Displayed correctly
Image selection and preview with Analyze and Change Image buttons	Successful
Prediction of crop name, category tag and confidence	Displayed correctly
Nutrition tiles, About, Benefits, Uses, Did you know? and related varieties	Displayed correctly
About Us page (three information cards)	Displayed correctly
Local execution without an external service	Successful

Table 6.1: Application Verification Results

A sample test was carried out with a photograph of white pulse seeds scattered on a red surface. The application's output for this image is given in Table 6.2, and the corresponding screenshots are included in Chapter 7.

Output Field	Value Displayed
Predicted crop	Whole White Gram
Category	Pulse
Confidence	98%
Energy	341 kcal per 100 g
Protein	25 g
Carbohydrates	58 g
Fiber	18 g
Fat	1.4 g

Table 6.2: Sample Prediction Output

6.2 RESULTS DISCUSSION

The sample test shows that the complete pipeline works from end to end. The image was uploaded and previewed, preprocessed by the server, classified by the CNN, and the predicted crop was displayed together with a confidence of 98% and its nutrition information. The result page presents the information in clearly separated sections, so the user can move from "what is this?" to "what does it offer?" without leaving the page.

The interface is simple enough for a non-technical user: a single upload area, a preview and one Analyze button. Because the model runs in a local Flask server, the application can be used without any external prediction service.

However, a successful example, even with a high confidence score, does not establish the classification accuracy of the model. Reliable performance figures require evaluation on a held-out test dataset, as described in Section 6.3.

6.3 PERFORMANCE EVALUATION

The following metrics are used to evaluate the classification model.

Metric	Description	Value
Accuracy	Proportion of correctly classified test images	71.71%
Precision	Proportion of predicted positive cases that are correct for a class	74.69%
Recall	Proportion of actual positive cases correctly identified for a class	71.71%
F1-Score	Harmonic mean of precision and recall	71.92%
Confusion Matrix	Correct and incorrect classifications across the 38 classes	92.54%

Table 6.3: Evaluation Metrics

6.4 LIMITATIONS

The current application has the following limitations.

- Predictions are limited to the 38 varieties represented by the model; any other item will be forced into one of these classes.
- Prediction quality depends on image quality, lighting, background and similarity to the training data.
- Photos containing several different grains mixed together may give unreliable results; single-item photos work best.

- Visually similar varieties, such as different split pulses, may be misclassified.
- A high confidence score does not guarantee a correct prediction.
- Nutrition values are indicative reference values per 100 g and can differ for a specific sample.
- The application does not assess quality, grade or adulteration.

6.5 CONCLUSION

CropTag uses a custom CNN, trained from scratch, to identify 38 varieties of grains, millets and pulses from a single photograph. The trained V2 model is served through a local Flask inference server, and a web interface lets the user upload an image, analyze it and view the predicted crop, its confidence and its nutrition profile together with benefits, uses and related varieties.

The project demonstrates the practical application of deep learning and computer vision to food and agriculture and shows how a from-scratch model can be turned into an accessible, user-friendly tool for everyday kitchens.

6.6 FUTURE ENHANCEMENTS

- Evaluate the model on a held-out test set and photographs taken under real-world conditions, and report class-wise metrics.
- Extend the dataset to more varieties and more varied backgrounds and lighting.
- Compare the custom CNN with transfer-learning architectures.
- Display the top three predictions and warn the user when confidence is low.
- Detect and separate mixed samples containing several grains.
- Add live camera capture, a mobile application and regional-language support.
- Extend the system towards quality grading and adulteration detection.

CHAPTER 7
APPENDIX

7.1 SAMPLE CODE

The code below shows the inference pipeline of a CropTag-style Flask server: loading the trained model, preprocessing the image, predicting the class and returning the result.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import tensorflow as tf
import numpy as np
from PIL import Image
import os

app = Flask(__name__)
CORS(app)

MODEL_PATH = os.path.join(
    os.path.dirname(__file__),
    "saved_model_experiment",
    "best_model_experiment.keras"
)

IMG_SIZE = 160

CLASS_NAMES = [
    "Barley", "Black Rice", "Bridegroom Rice", "Maize",
    "Oats", "Paddy", "Parboiled Rice", "Ponni Rice",
    "Red Rice", "Seeraga Samba Rice", "Wheat",
    "Wild Elephant Rice", "Barnyard Millet", "BrownTop Millet",
    "Finger Millet", "Foxtail Millet", "Kodo Millet",
    "Little Millet", "Pearl Millet", "Proso Millet", "Quinoa",
    "Red Sorghum", "White Sorghum", "Black Chickpeas",
    "Black Eyed Pea", "Black Gram", "Chana Dal", "Field Bean",
    "Green Gram", "Horse Gram", "Mysore Dal", "Peas", "Rajma",
    "Soya Bean", "Split Moong Dal", "Thor Dal",
    "White Chickpeas", "Whole White Gram"
]

CATEGORY_NAMES = [
    "Grain",
    "Millet",
    "Pulse"
]

print("Loading V3 Experiment model...")
model = tf.keras.models.load_model(MODEL_PATH)
print("V3 Experiment model loaded successfully.")
```

```
@app.route("/")
def home():
    return jsonify({
        "status": "running",
        "model": "GrainScan AI V3 Experiment"
    })

@app.route("/predict", methods=["POST"])
def predict():

    if "image" not in request.files:
        return jsonify({"error": "No image uploaded"}), 400

    try:
        image = Image.open(
            request.files["image"].stream
        ).convert("RGB")

        image = image.resize((IMG_SIZE, IMG_SIZE))

        image_array = np.asarray(
            image,
            dtype=np.float32
        )

        image_array = np.expand_dims(
            image_array,
            axis=0
        )

        predictions = model.predict(
            image_array,
            verbose=0
        )

        class_output = np.asarray(
            predictions["class_output"]
        )

        category_output = np.asarray(
            predictions["category_output"]
        )

        print("Class output shape:", class_output.shape)
        print("Category output shape:", category_output.shape)

        class_probabilities = class_output[0]
        category_probabilities = category_output[0]
```

```
        predicted_class_index = int(
            np.argmax(class_probabilities)
        )

        predicted_category_index = int(
            np.argmax(category_probabilities)
        )

        return jsonify({
            "prediction": CLASS_NAMES[predicted_class_index],
            "confidence": round(
                float(class_probabilities[predicted_class_index]) * 100,
                2
            ),
            "class_index": predicted_class_index,
            "category": CATEGORY_NAMES[predicted_category_index],
            "category_confidence": round(
                float(category_probabilities[predicted_category_index]) * 100,
                2
            )
        })

    except Exception as e:
        print("PREDICTION ERROR:", repr(e))

        return jsonify({
            "error": str(e)
        }), 500

if __name__ == "__main__":
    print("Starting V3 Experiment server...")
    print("Open http://127.0.0.1:5002")

    app.run(
        host="127.0.0.1",
        port=5002,
        debug=False
    )
```

Note: the `LABELS` list must contain the same 38 class names, in the same order, that were used when the model was trained.

Class Name:

1. Grains — 12 classes

1. Barley
2. Black Rice
3. Bridegroom Rice
4. Maize
5. Oats
6. Paddy
7. Parboiled Rice
8. Ponni Rice
9. Red Rice
10. Seeraga Samba Rice
11. Wheat
12. Wild Elephant Rice

2. Millets — 11 classes

13. Barnyard Millet
14. BrownTop Millet
15. Finger Millet
16. Foxtail Millet
17. Kodo Millet
18. Little Millet
19. Pearl Millet
20. Proso Millet
21. Quinoa
22. Red Sorghum
23. White Sorghum

3. Pulses — 15 classes

24. Black Chickpeas
25. Black Eyed Pea
26. Black Gram
27. Chana Dal
28. Field Bean
29. Green Gram
30. Horse Gram
31. Mysore Dal
32. Peas
33. Rajma
34. Soya Bean
35. Split Moong Dal
36. Thor Dal
37. White Chickpeas
38. Whole White Gram

Total: 38 classes — 12 Grains + 11 Millets + 15 Pulses.

7.2 OUTPUT SCREENSHOTS



Fig. 7.1: CropTag home page and image upload interface



Fig. 7.2: About Us page

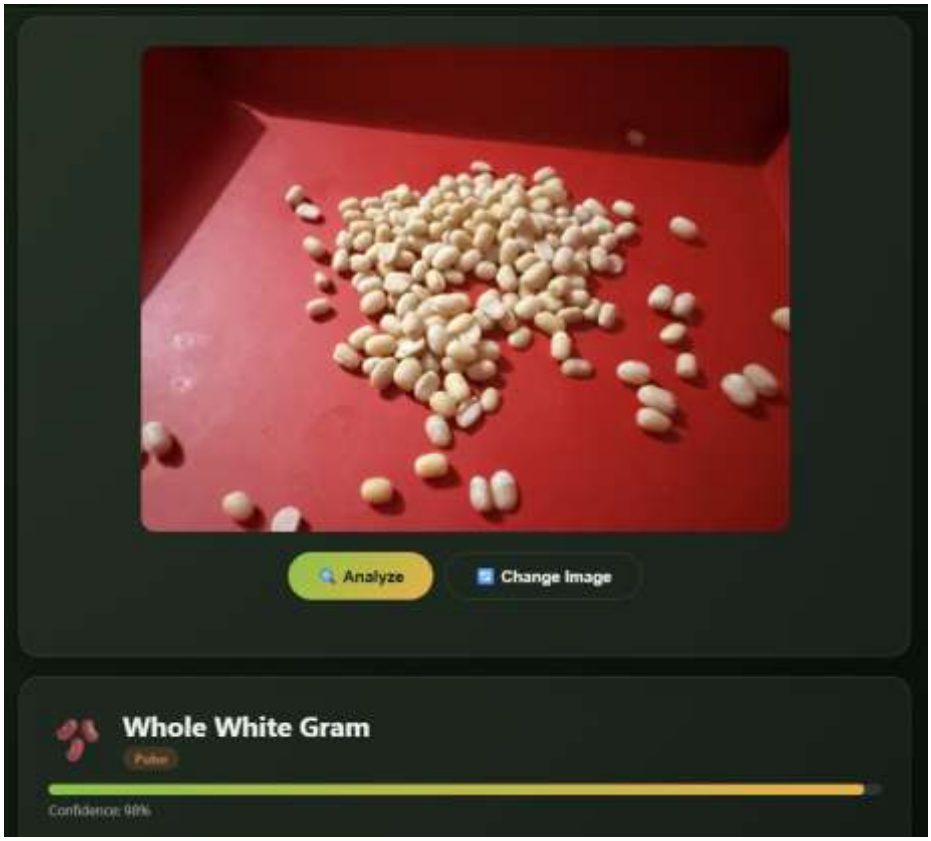


Fig. 7.3: Image selected for analysis with predicted crop and confidence score

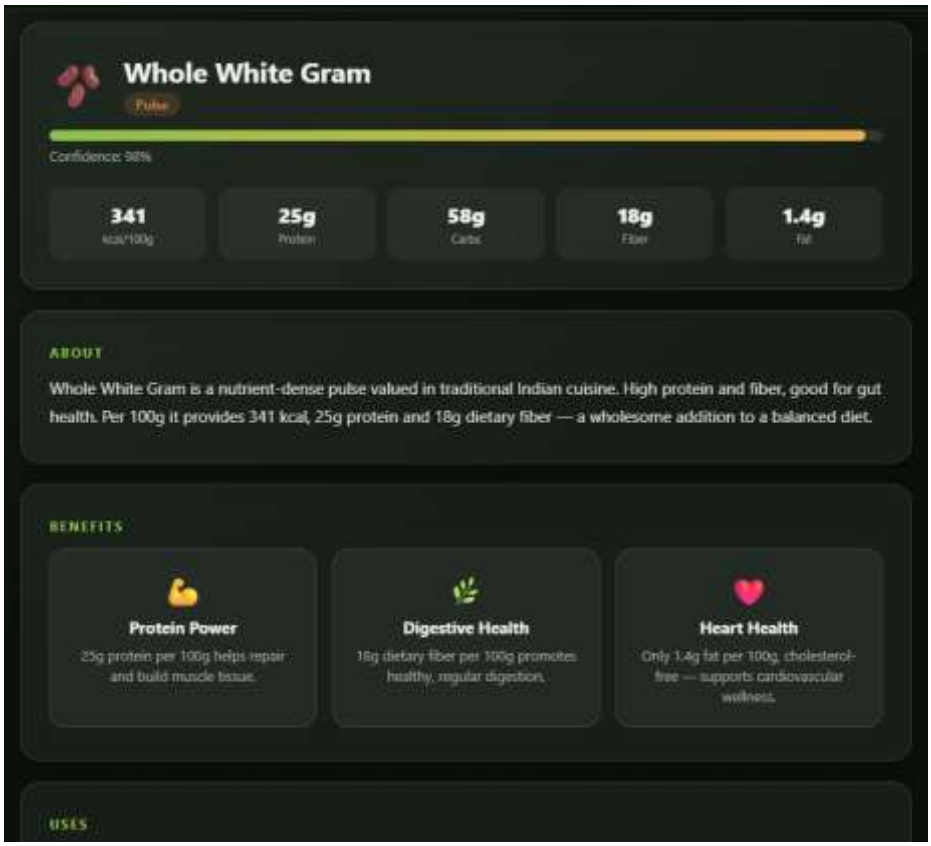


Fig. 7.4: Nutrition profile, About and Benefits sections of the result

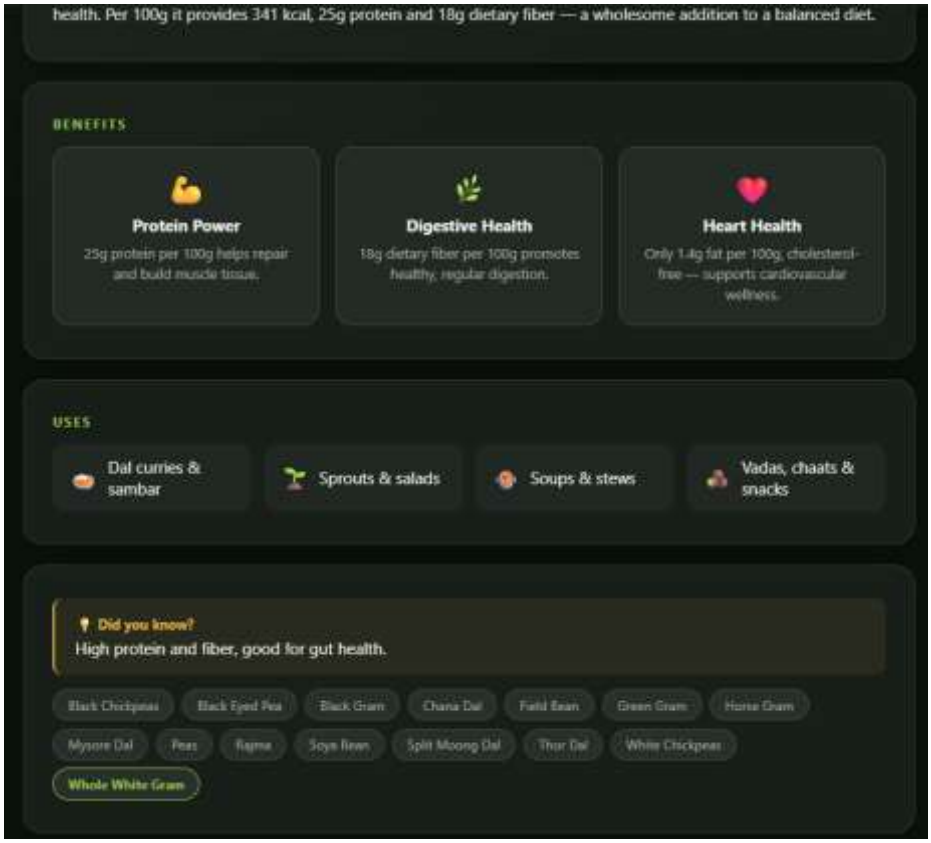


Fig. 7.5: Benefits, Uses, Did you know? and related varieties

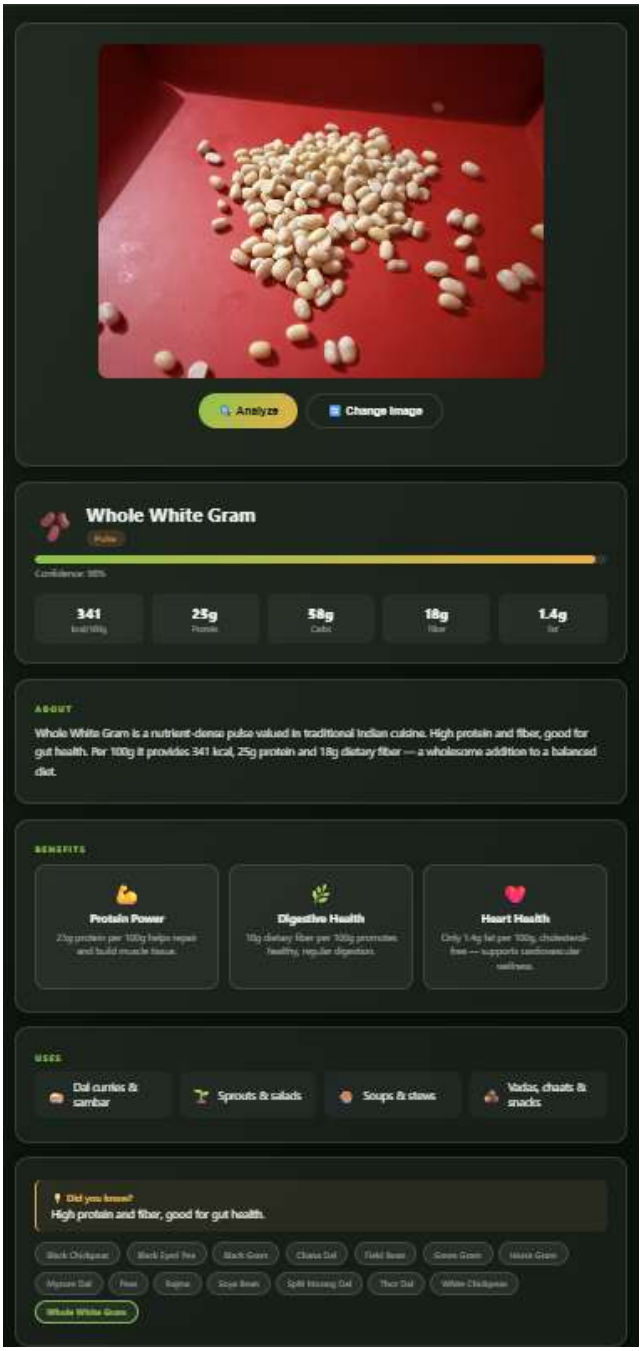


Fig. 7.6: Complete result page

REFERENCES

[1] TensorFlow. TensorFlow Documentation. Available at: https://www.tensorflow.org/

[2] Keras. Keras Documentation. Available at: https://keras.io/

[3] Pallets Projects. Flask Documentation. Available at: https://flask.palletsprojects.com/

[4] NumPy. NumPy Documentation. Available at: https://numpy.org/doc/

[5] Pillow. Pillow Documentation. Available at: https://pillow.readthedocs.io/

[6] Mozilla. MDN Web Docs (HTML, CSS and JavaScript). Available at: https://developer.mozilla.org/

[7] Kaggle URL: https://www.kaggle.com/datasets/arunthiraviam/indian-grains-pulses-and-millets-dataset